

简易 BUCK 降压变换器

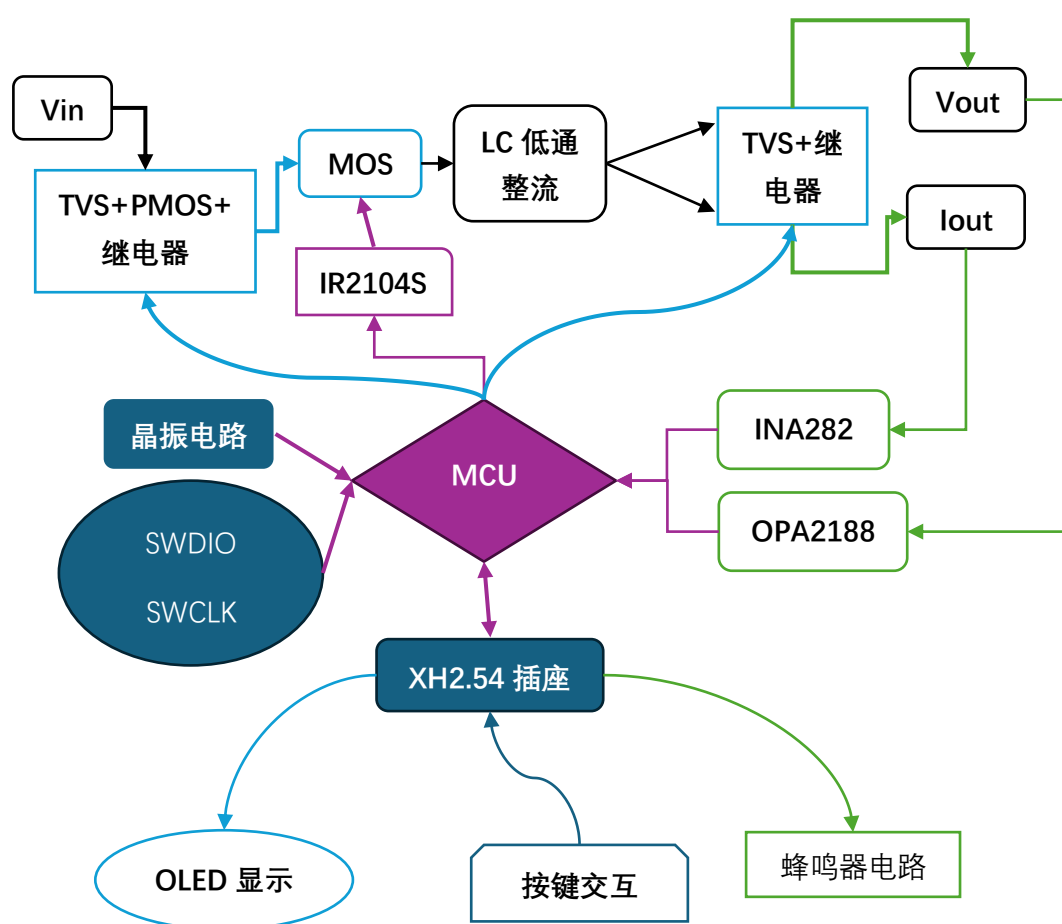
——by 24 集成 张镜涛

一、总体方案

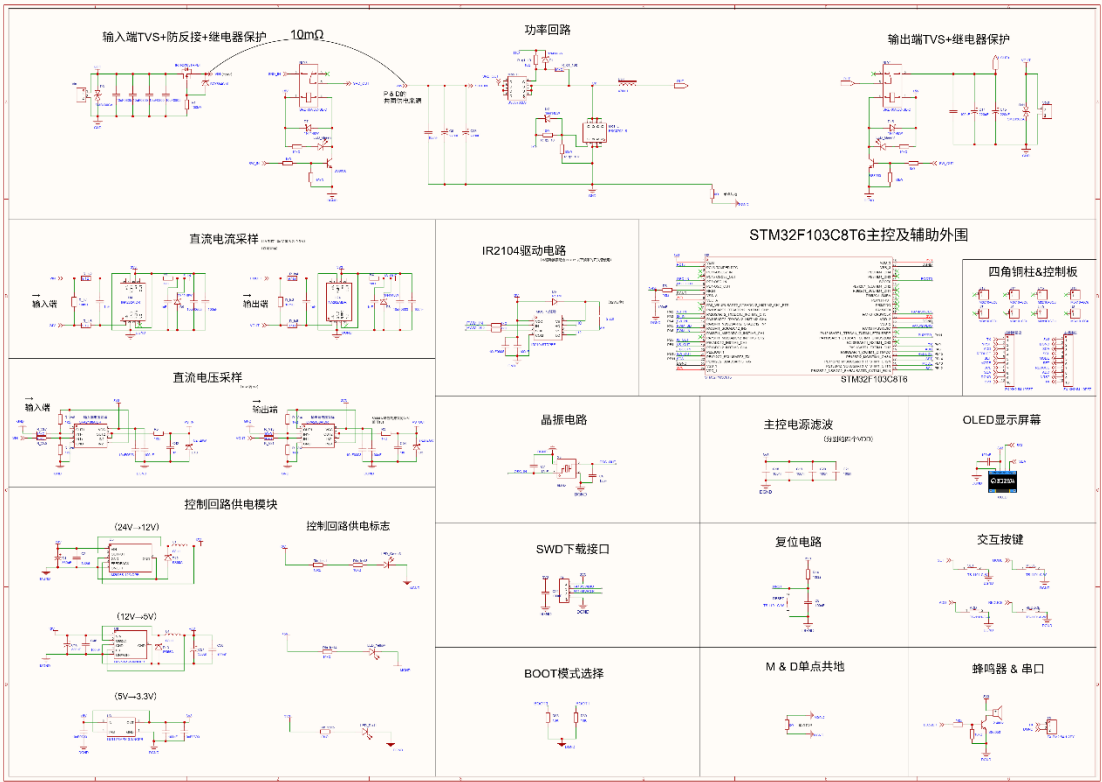
功率回路采用同步整流 **BUCK** 拓扑结构，主控制系统使用 **stm32f103c8t6** 控制芯片，MCU 由 **24V 输入降压** 并且稳压至 **3.3V** 供电，控制回路供电系统、AD 采样芯片均使用 **TI** 芯片，栅极驱动芯片为 **IR2104S**，输入级采用 **PMOS 防反接保护**、**继电器** 保护以及 **TVS** 静电保护，输出级采用 **继电器** 保护以及 **TVS** 静电保护，软件过流保护通过控制**两级的继电器**和同步 MOS 的栅极驱动器以实现，外接控制板作为输入输出状态显示以及操控，核心控制算法为**串级双 PI** 控制。

说明：同步整流的启动开关，输出选择等控制操作均由外接控制板实现，输出电压可调范围为 **5.00V – 19.00V**（24V 输入），在 **12V±4V** 以外的输出区间存在微小的失真。

原理框架

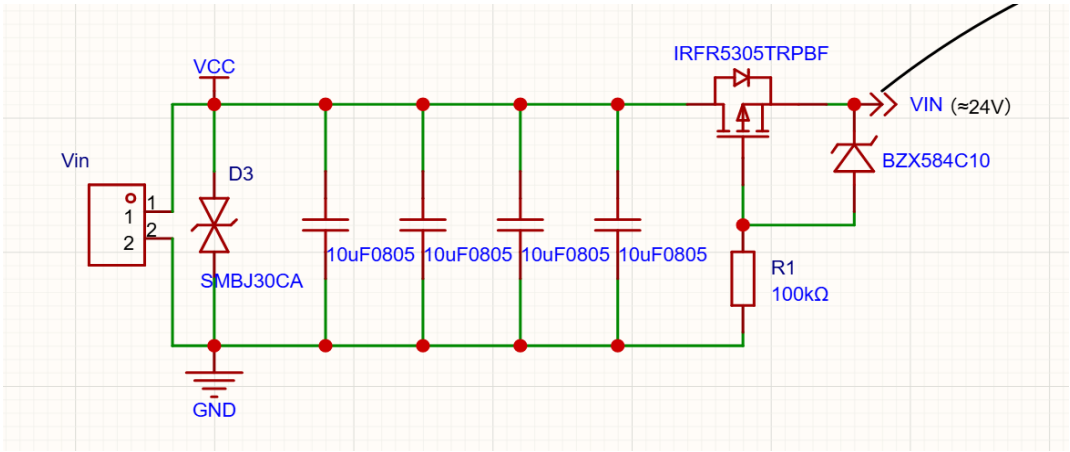


原理图



二、硬件详解

1. 输入级 PMOS 防反接

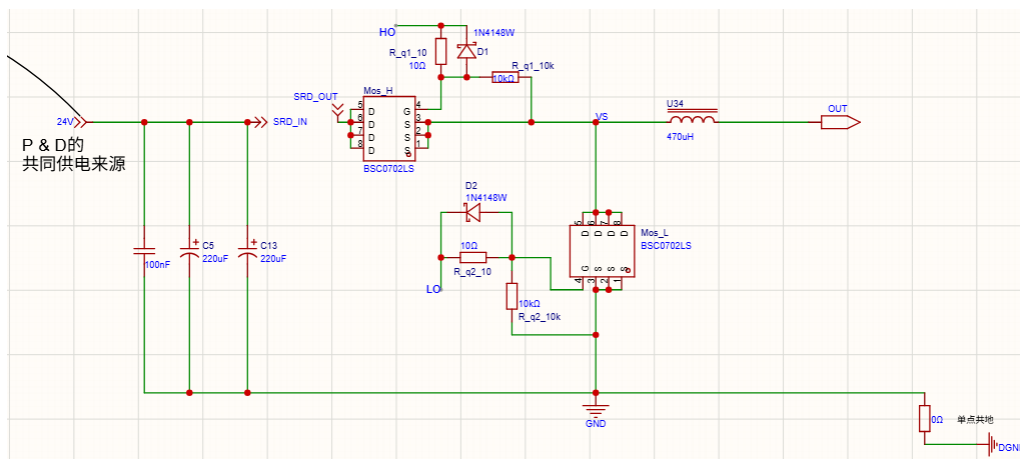


核心元件：稳压二极管 BZX584C10（典型反向钳位电压 10V，[反向漏电流典型值为 0.2uA@7V](#)）；双向 TVS SMBJ30CA（D3），30V 截止；PMOS 管 IRFR5305（耐压 55V，电流 31A，典型值 $R_{ds(on)}$ 65mΩ @ -10V， $V_{gs(th)}$ =-2V）。

工作原理

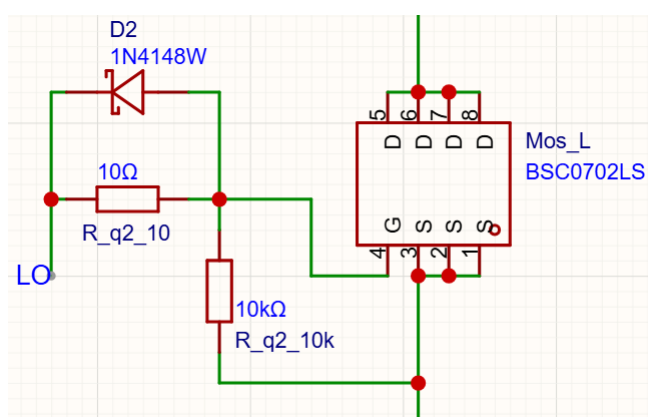
1. 正向输入时，上端 VCC、下端 GND，如图所示，输入先通过 PMOS 的体二极管正

说明：4 个并联的前级电容用于抑制上电瞬间的尖峰电压，防止对 PMOS 造成损伤；电容选型不可选用电解电容，否则反接时有炸容风险。



核心元件： NMOS 管 BSC0702LS（典型值 V_{ds} 60V, $R_{ds(on)}$ 2.7m Ω @ 10V, $V_{gs(th)}$ 1.7V, Q_g 24nC), 功率电感 ($L = 470\mu\text{H}$, $\text{DCR} = 76\text{m}\Omega$, 线径 $D=1\text{mm}$, 可过最大电流 10A), 输入输出电容 $4 \times 220\mu\text{F}$ (输出电容图中未显示) 滤除低频噪声 + $2 \times 100\text{nF}$ 滤除高频噪声, 0Ω 电阻单点共地区分功率地和数字地, 二极管 1N4148W (100V @ 150mA), 10Ω & $10\text{k}\Omega$ 电阻。

工作原理： Mos_H 和 Mos_L 上管下管轮流工作, 同步切换, 配合电感 L 和输出电容 C (输出电容 C 图中未显示) 整流实现 DC-DC 降压, 核心主要通过调整上管 Mos_H 的导通时间占比来调整输出, $V_{out} = \frac{V_{in} \cdot t_{on} + 0 \cdot t_{off}}{T}$, $T = t_{on} + t_{off}$, 很明显 t_{on} 越大, 输出电压越大。于是通过调节占空比 $\text{duty} = \frac{t_{on}}{T}$, 便可以调整输出电压。



说明： 由于 MOS 管 GS 存在寄生电容 C_{gs} , V_{gs} 短时间无法快速变化导致 MOS 管无法快速开关, 故需要二极管 D2 (1N4148W) 用于 MOS 关闭时“快速泄流”, 加速关闭; 10Ω 电阻 R_{q2_10} 为输入电阻, 防止栅极驱动灌电流过大损坏栅极 G; $10\text{k}\Omega$ 电阻 R_{q2_10k} 主要用于钳位 V_{gs} (开启状态) 和下拉 V_g (关闭状态), 防止 V_g 相对 V_s 浮空导致 MOS 管误开关或者处于半开关的高阻态而损坏 MOS 管。

4. buck 拓扑电感 L 和输出电容 C 选型

开关频率 $f_{sw} = 20\text{kHz}$, 输入电压 $V_{in} = 24\text{V}$, 目标输出电压 $V_{out} = 12\text{V}$, 占空比 $D = 0.5$, 目标电流纹波 ΔI 控制在 0.80A (纹波率约为 40% @ 2A), 则电感

$$L = \frac{(V_{in} - V_{out})D}{f_{sw} \cdot \Delta I}$$

可得电感 $L = 375\mu\text{H}$, 取 $L = 470\mu\text{H}$, 目标输出电压纹波 $\Delta V \leq 1\text{V}$, 则

$$\Delta V = \frac{\Delta I}{8 \cdot f_{sw} \cdot C} + \Delta I \cdot ESR$$

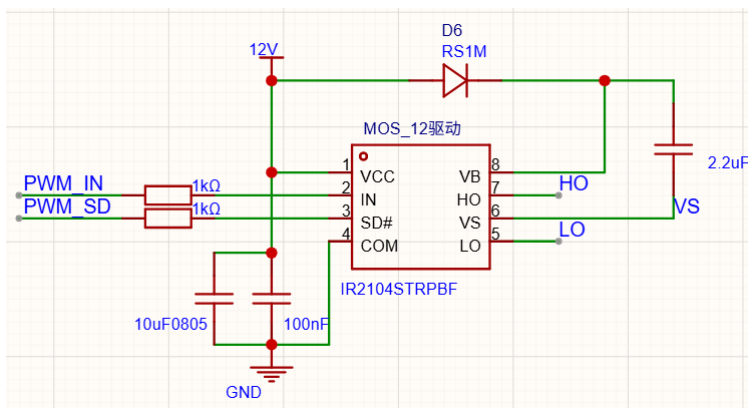
由此可得

$$\Delta V = \frac{1}{200000 \cdot C} + 0.8 \cdot ESR$$

大多 $100\mu\text{F}$ 以上的电容寄生电阻 ESR 均在 $100\text{m}\Omega$ 以上, 本次采用双 $220\mu\text{F}$ 电容并联作

$$ESR = \frac{DF}{2\pi fC}$$

5. IR2104S 驱动电路



工作原理：IR2104S 内部存在一个推挽电路，能在短时间内提供较大的灌拉电流，帮助 MOS 管的 GS 的寄生电容 C_{gs} 快速充放电，让 V_{gs} 快速升至 $V_{gs(th)}$ 以上或快速降至 $V_{gs(th)}$ 以下来加速 MOS 管的开关，防止 MOS 长期处于半开关的高阻态而烧坏。同时 IR2104S 内置死区时间 deadtime 卡在上管和下管切换导通的区间内，可以有效防止上下管切换时上管还未完全关闭导致的上下管直通接地短路而烧坏。IR2104S 内置自动生成 PWM_IN 互补的信号输出给 LO，用于同步整流。

说明：输入信号 PWM_IN 和 PWM_SD 由主控 stm32f103c8t6 提供，PWM_SD 置高电平则启动驱动输出，低电平则关闭驱动输出（IR2104S 兼容 3.3V 逻辑电平）；由于上 MOS 管导通后 $V_s = V_{in} \text{ (typical 24V)} > V_g \text{ (typical 12V)}$ 会导致 $V_{gs} < V_{gs(th)}$ ，故需要利用电容两端电压不突变的特性，让自举电容 Cboot (2.2uF) 抬高上管的 V_{gs} ，最后配合自举二极管 RS1M 防止电流倒灌（VS (24V)，驱动芯片 VCC (12V)）。

6. IR2104S 自举二极管&自举电容选型

同时由于本项目 BUCK 整流的开关频率为 20kHz (开关周期 50us = 50000ns)，故对自举二极管的反向恢复时间无高要求，使用 RS1M (反向恢复时间 500ns) 即可。

自举电容选型：需确保电容充电时间大于上管关闭的时间，上管下一个周期可以正常处于导通状态；本次选用 VCC 为 12V，MOS 管的开启电压 $V_{gs(th)}$ 典型值 1.7V、最大值为 2.3V，RS1M (V_f 大约 1.3V~1.5V)，取 $V_{bootstrap} = 8.2V$ ，故允许跌落的电压

$$\Delta V = V_{CC} - V_f - V_{bootstrap}$$

故 $\Delta V = 2.3V$ ；MOS 管 Q_g 典型值为 24nC， Q_{total} 取 Q_g 的 10 倍，故最小电容

$$C_{boot_min} = \frac{Q_{total}}{\Delta V}$$

可得 $C_{boot_min} = 104.35 \text{ nF}$ ；BUCK 开关频率 20kHz，最大占空比约为 89%

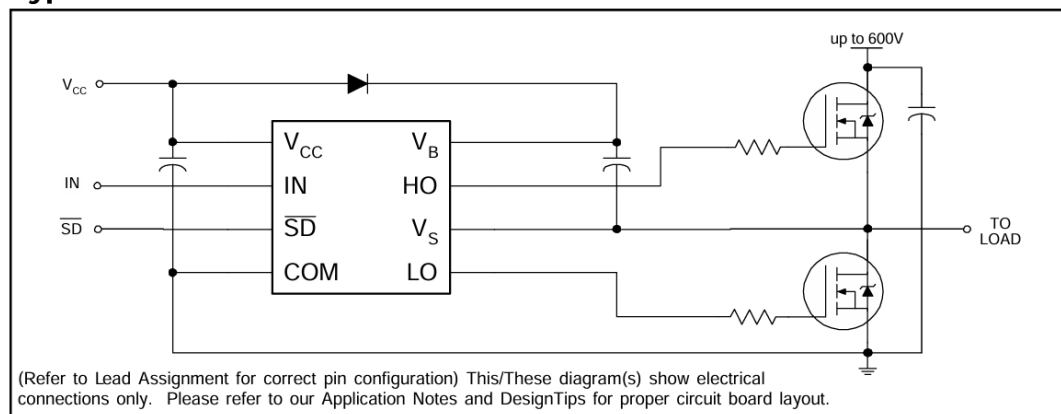
(24V->19V，留一定余量)，故最短充电时间为 $\Delta t = 5.5\mu s$ (5.5us 内把需要把 ΔV 补回来)，假设实际通过 RS1M 电流近乎稳定 $I = 1A$ (实际更大)，故最大电容

$$C_{boot_max} = \frac{I \cdot \Delta t}{\Delta V}$$

故 $C_{boot_max} = 2.39 \mu F$ ；故自举电容 C_{boot} 的取值范围大致为 104.35nF ~ 2.39uF，本次项目实际绝对大多数占空比使用均小于 80%，且计算时多次留取一定的余量，故 C_{boot} 可取较大数值 **2.2uF** (本次外围电路设计完全参考 IR2104S 数据手册，故缺乏 R_{boot} 设计，上电初期部分分压由功率回路承担，校准后不影响正常工作输出)。

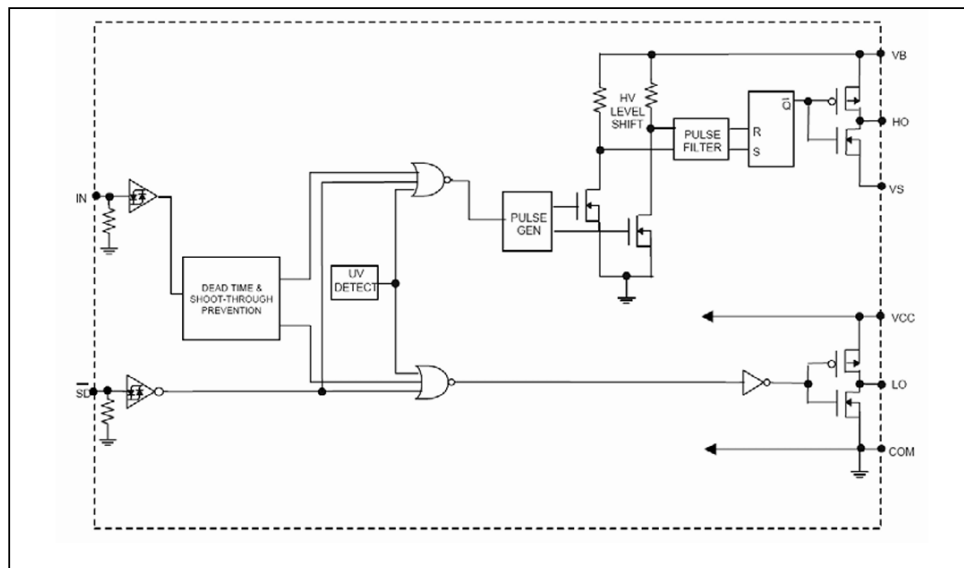
官方参考设计图

Typical Connection

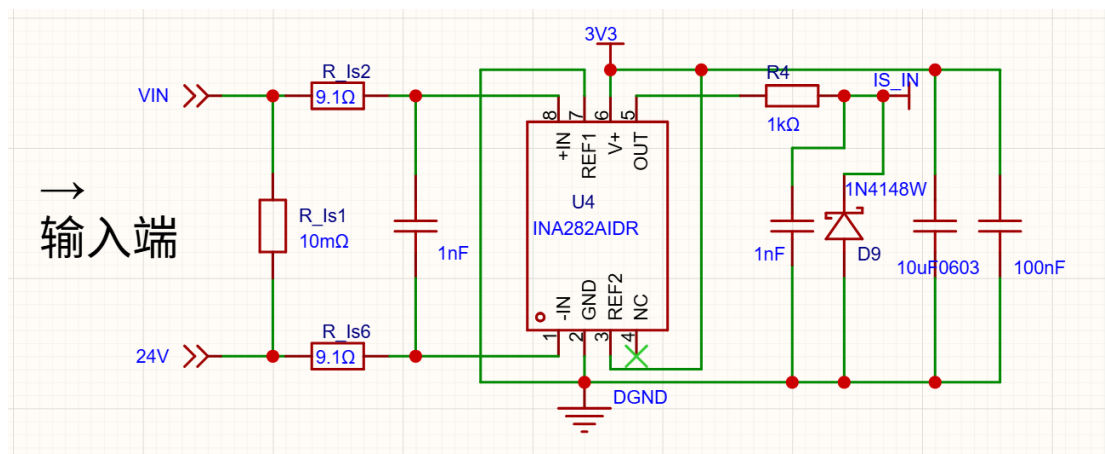


IR2104S 内部框图

Block Diagram



7. 直流电流采样（以输入级为例）

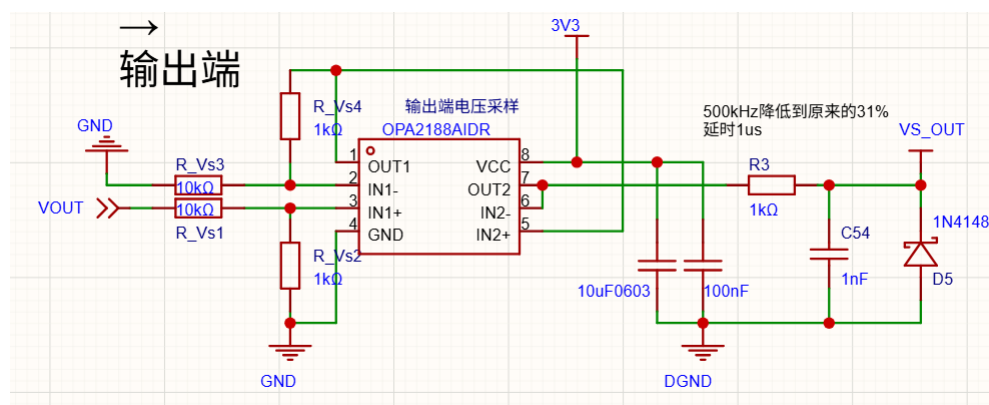


核心元件：INA282AIDR 双向电流放大芯片（-14V~80V 输入，有效带宽 10kHz，TI 芯片），10mΩ采样电阻，二极管 1N4148W（100V @ 150mA），输出 RC（1kΩ+1nF）低通滤波。

工作原理：将 10mΩ采样电阻串联入功率回路，流过电阻的电流为 I ，则输入电压差为 $I \times R_{Is1}$ ，再通过内部运算放大器将差模信号放大 50 倍输出，故 $V_{OUT} = 0.5V / A$ 。

说明：供电电压为与 MCU 同电压 3.3V，可以有效限制输出电平在 3.3V 左右，即使采样信号超出量程，也不至于对 MCU 的 AD 采样引脚造成损害；本次开关频率为 20kHz，采样芯片带宽为 10kHz，对采样信号的交流分量有较好的抑制作用；输出 RC 对 500kHz 以上的噪声能削减至 31%，效果较好，但信号输出有 1μs 的滞后。

8. 电压采样（以输出级为例）



核心元件：OPA2188 运算放大器（TI），R_Vs1、R_Vs2、R_Vs3、R_Vs4 分压电阻，二极管 1N4148W（100V @ 150mA），输出 RC（1kΩ+1nF）低通滤波。

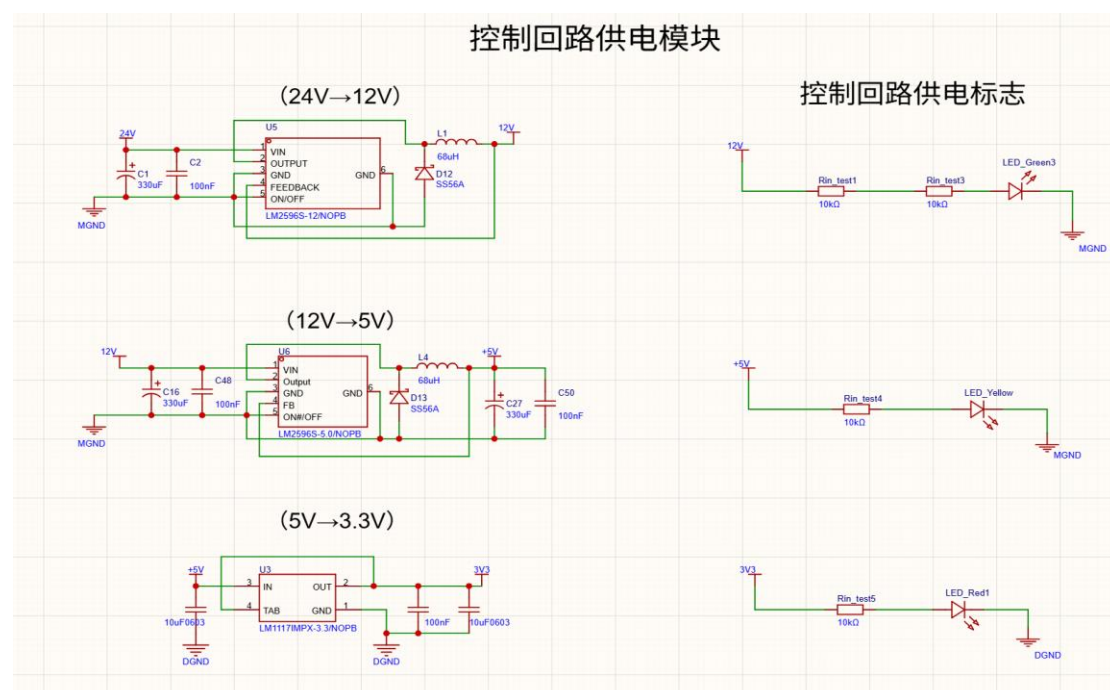
工作原理：通过电阻分压将电压信号削弱至原本的 1/11 输入，利用运算放大器负反馈“虚短、虚断”特性，则有

$$V_{out} \frac{R_{Vs2}}{R_{Vs1} + R_{Vs2}} = V_{out1} \frac{R_{Vs3}}{R_{Vs3} + R_{Vs4}},$$

故 $V_{out1} = V_{out} / 10$ ，后级第二路运算放大器则用于电压跟随，隔离输出，故 $V_{out2} = V_{out1} = V_{out} / 10$ 。

说明：与电流芯片同理，供电电压也为与 MCU 同电压得 3.3V；同样地，输出 RC 对 500kHz 以上的噪声能削减至 31%，信号输出有 1us 的滞后。

9. 控制、采样芯片得供电系统

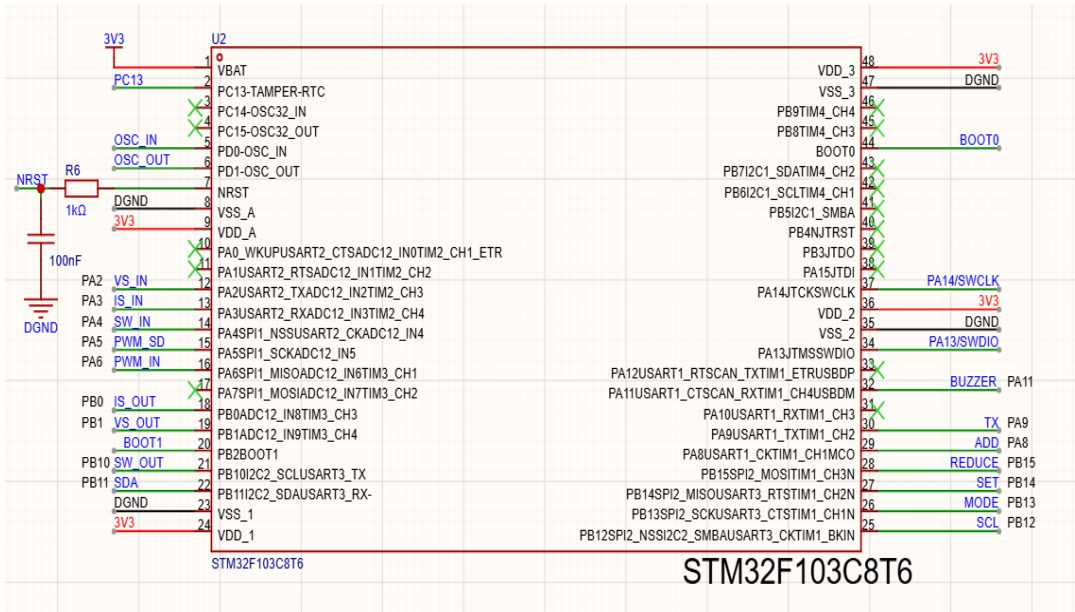


核心元件：双 DC-DC 芯片 LM2596 (TI)，LDO LM1117 (TI)，100uH、68uH 电感。

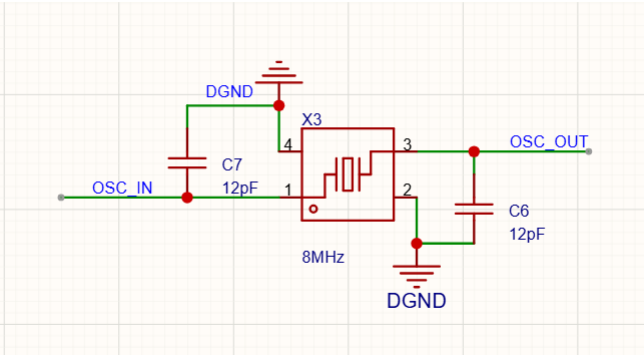
工作原理：逐级降压，逐级供电，24V->12V，为需要较大电压电流的栅极驱动芯片 IR2104S 供电 12V；12V->5V，同时为继电器供电；最后用 LDO 稳压至 3.3V 给需要精准 VCC 的电压&电流采样芯片、MCU 供电，同时也为蜂鸣器供电。

说明：本次供电带 3 个 LED 指示灯，当这三级电源正常供应时，3 个 LED 均正常发光提供指示；前两级压差过大，采用 DC-DC 芯片降压，最后一级需要稳定电平，采用 LDO 稳压。

10. MCU 引脚布局分配及其外围



说明：stm32f103c8t6 片上外设资源较多，尽量按照 PCB 布局分配引脚，防止信号线交叉干扰等问题；NRST 为复位信号接收引脚，为防止受干扰，需要在引脚处设置低通滤波器，防止 MCU 受到干扰信号时离奇复位。



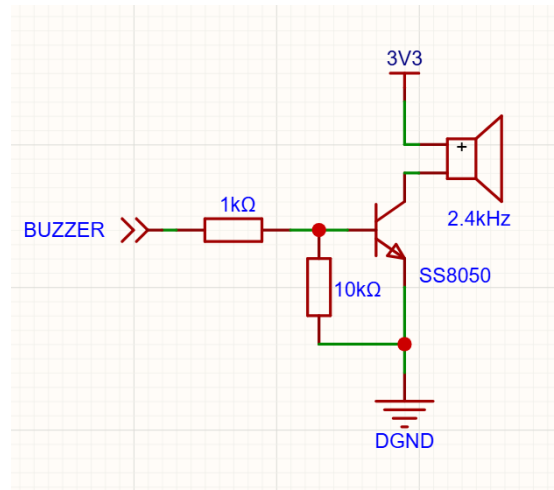
晶振电路：外置输入晶振，提供更加准确稳定的时钟信号。

说明：作为 MCU 的核心高频时钟信号源，PCB 布局需离 OSC 引脚尽可能近，防止受到外界干扰。

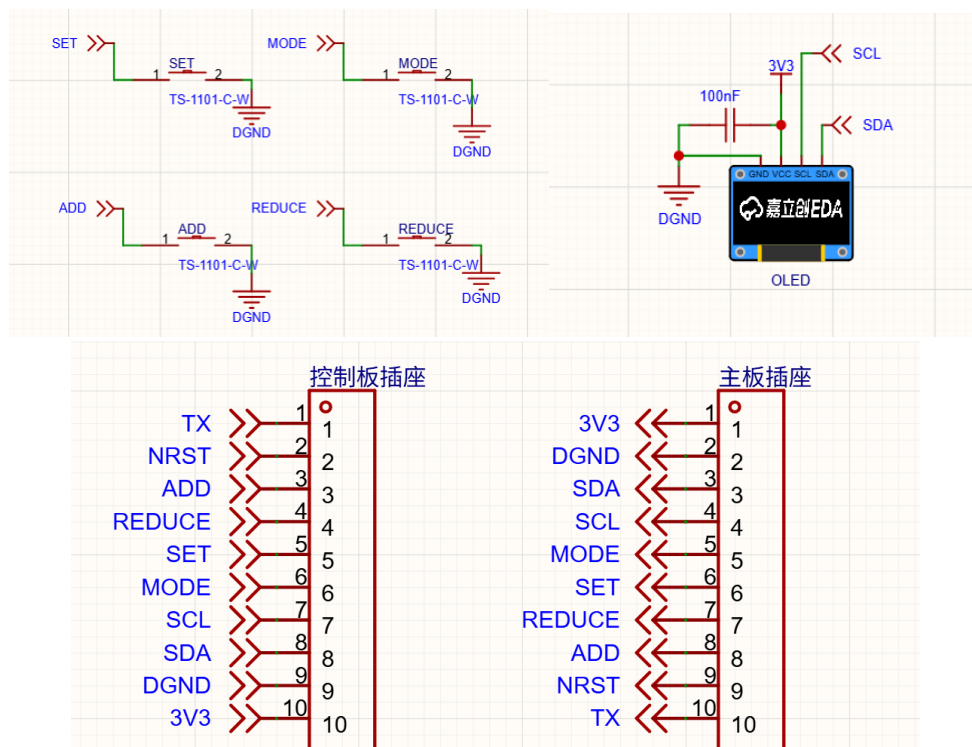
蜂鸣器电路

BUZZER 信号由 MCU 提供，通过三极管控制蜂鸣器的开启与关闭，从而提示电路状态。

说明：10Ω电阻：用于钳位（导通时）与下拉（截止时），主要防止上电初期，基极 B 处于悬空状态电平不定导致蜂鸣器误触发启动。



控制板



说明：信号均由 MCU 提供，通过 XH2.54 插座和 10p 端子线引出主板外进行远程控制操作的实现，同时带有 OLED 实时刷新显示，监控主板的输入输出情况。

三、MCU 主要算法

1. 串级 PI 控制

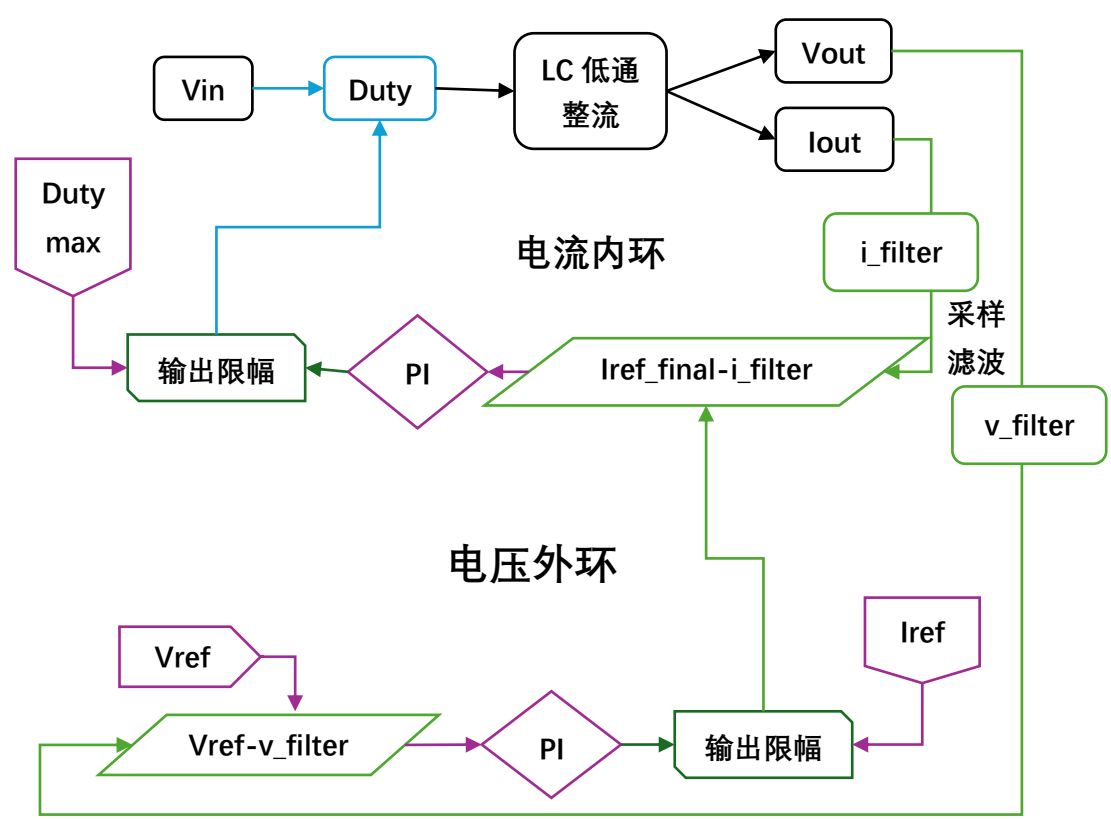
说明：使用 stm32f103c8t6 自带的 TIM3 定时器设置输出 PWM 信号，并且每个周期进行一次中断控制，计算需要输出的 duty 值进行预装载，并在下一个 PWM 周期生效。

TIM3 关键参数配置

```
/*配置时钟源*/
TIM_InternalClockConfig(TIM3); //选择 TIM3 为内部时钟

/*时基单元初始化*/
TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure; //定义结构体变量
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1; //时钟分频，选择不分频，此参数用于配置滤波器时钟，不影响时基单元功能
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up; //计数器模式
TIM_TimeBaseInitStructure.TIM_Period = 3600 - 1; //计数周期，即 ARR 的值
TIM_TimeBaseInitStructure.TIM_Prescaler = 1 - 1; //预分频器，即 PSC 的值
//值(ARR+1)*(PSC+1)=72M/FreqTarget
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0; //重复计数器，高级定时器才会用到
TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure); //将结构体变量交给 TIM_TimeBaseInit，配置 TIM3 的时基单元
```

PI 双环控制



电流内环

```
//电流环
//Pi
duty_p = (Iref_final - i_filter) * 78 / 100 ; //0.78
//Ii
if(i_filter > Iref_final + 5) duty_integral -- ;
if(i_filter < Iref_final + 5) duty_integral ++ ;
else;
//积分限幅
if(duty_integral >= 3200) duty_integral = 3200;
if(duty_integral <= -360) duty_integral = -360;

duty = duty_p + duty_integral;
//总输出限幅保护，避免 PWM 占空比 溢出
if(duty >= 3200) duty = 3200;
if(duty <= 0) duty = 0;

PWM_Duty_Set(duty);
```

电流内环通过计算每个周期电流采样值 i_filter 与电流目标值 $Iref_final$ 的误差值 $(Iref_final - i_filter)$ 来输出 **duty_p**，主打一个“缺多少补多少”的思路，把实际的输出电流值 I 牢牢控制在 $Iref_final$ 的位置。

这套说法表面天衣无缝，但仔细分析又会发现其实经不起推敲：当误差值为 0 时，**duty_p** 的输出值便为 0，若单靠 **duty_p** 输出，在刚达到目标电流的一瞬间，误差为 0，所以输出也为 0，没有了持续稳定的 **duty** 输出，电流值便会快速下降，而电流值一旦下降，误差 $(Iref_final - i_filter)$ 就又不复为 0，于是 **duty_p** 又会重新输出……这样反复折腾，最终输出的电流值 I 会收敛于目标电流值 $Iref_final$ 下方的位置，而不是目标电流值 $Iref_final$ 。

于是这时 **duty** 的另一个输出分量 **duty_integral（积分量）** 就发挥作用了，不像比例分量 **duty_p** 那样的“粗野”，积分量更像一个妥妥的“细准焦螺旋”，它通过不断比较采样值与目标值的大小，来对输出值进行 +1、-1 这种细微的操作，只要采样值 i_filter 一直卡在 $Iref_final$ 下方，他就会不停的 +1+1+1……不停地累加输出量，直到 **$i_filter = Iref_final$** 为止。

有了比例分量 **duty_p** 和积分量 **duty_integral**，我们只需把两者相加结合共同输出，便可以把输出值控制在我们想要的位置 $Iref_final$ ，这便是电流环，能够实现**对电流的精准把控**。

但是我们的输出不仅要控制电流，还要控制电压，于是便有了电压外环。

电压外环

```
//电压环
voltage_count ++;
if(voltage_count == 10)
{ voltage_count = 0;
//Pv
Iref_final_p = (Vref - v_filter) * 37 / 250; // 0.148

//Iv
if(v_filter < Vref - 2) Iref_final_integral ++ ;
if(v_filter > Vref - 2) Iref_final_integral -- ;
else;

//积分限幅
if(Iref_final_integral >= Iref) Iref_final_integral = Iref;
if(Iref_final_integral <= -Iref) Iref_final_integral = -Iref;

Iref_final = Iref_final_p + Iref_final_integral;

//总输出限幅保护，避免 Iref_final 溢出
if(Iref_final >= Iref) Iref_final = Iref;
if(Iref_final <= -Iref) Iref_final = -Iref;
//参考值线性校准
Iref_final = 0.8696 * Iref_final + 0.7517;
} // if(voltage_count == 10)
```

与电流环一样，电压环也是一个 PI 控制器，通过不停地计算当前电压值与目标值的差距，来调控电压的输出，使其收敛于我们想要的目标值。

但是电压 PI 外环的输出值可不是占空比 duty，而是给电流的参考值 Iref_final，电压环的工作就是调节**电流内环**这个“**电流源**”的输出电流，由欧姆定理

$$V_{out} = I_{out} \cdot R_{load}$$

可得电压环自己需要的 **Vref**。

这一套控制的好处在于能同时控制 Vout 和 Iout，只要给电压环的输出位置加一个输出限幅 Iref，便可以限制系统最大的输出电流，而一个能限制最大输出电流的恒压源（也就是存在 CC 模式限流）才是一个理想又安全的恒压源，这相比于单电压环调控，安全性更高，相比双环切换方案（电压单环和电流单环切换着输出）更加丝滑，几乎不存在尖峰电压或者浪涌电流。

2. 输出采样与滤波

TIM 参数配置

```
/*配置时钟源*/
TIM_InternalClockConfig(TIM2); //选择 TIM2 为内部时钟，若不调用此函数，TIM2 默认也为内部时钟

/*时基单元初始化*/
TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure; //定义结构体变量
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1; //时钟分频，选择不分频
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up; //计数器模式
TIM_TimeBaseInitStructure.TIM_Period = 36 - 1; //计数周期，即 ARR 的值
TIM_TimeBaseInitStructure.TIM_Prescaler = 10 - 1; //预分频器，即 PSC 的值
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0; //重复计数器，高级定时器才会用到
TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure); //将结构体变量交给 TIM_TimeBaseInit，配置 TIM2 的时基单元
```

滤波计算

```
// 电压 Vout 采样迭代滤波
v_filter = (v_filter * 4 + AD_Value[3] * 6) / 10;

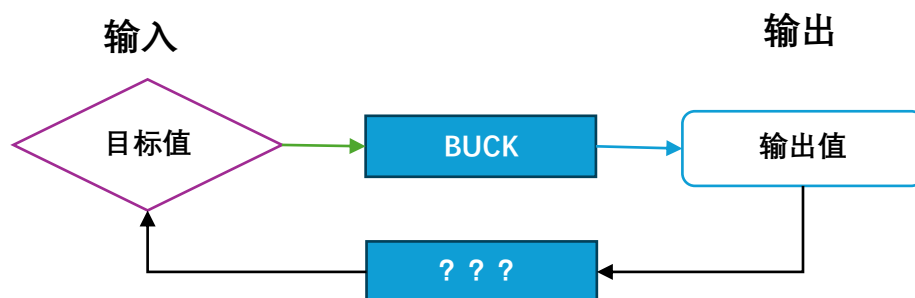
// 电流 Iout 采样迭代滤波
i_filter = (i_filter * 4 + (AD_Value[2]-2048) * 6) / 10;
```

滤波采用**迭代滤波**的方式，算法简洁，且可以有效抑制尖峰噪声等，但是响应较慢，存在一定的惯性，为了解决这个问题，我额外开启一个定时器 **TIM2** 进行单独的滤波计算，加快其计算频率，每 **1 个 PWM 周期**就足以让 filter **迭代 10 次**。

说明：**AD 采样**的周期大约为 **4.625us**，**滤波迭代**的周期为 **5us**，几乎与 AD 采样同频工作，将采样输出的利用率拉到最大。

3. 检验与教准

由于实物系统本身电路存在各种寄生参数，并且输入输出，回路等也存在一部分电阻，电平供应也无法确保完全准确无误，采样芯片采样的数据也难免受影响，实际的电路无法像理论分析那样完美，即便在有 PI 控制调节的情况下，误差也依然存在，为此，我们还需要做一部分的“校准工作”，为了方便理解，我们可以把整个系统模型进行抽象提取为一个整体，只看输入和输出，以便于分析。



我们每一次给系统输入一个**参考值** V_{ref} , I_{ref} 就可以让系统跑一遍, 然后获得一个**输出值**, 但是这个输出值往往并不准确, 输出的结果不等于我们输入的参考值, 所以我们现在要做的就是输入一个“特别的参考值”, 让他“流过系统”最后出来的时候变成“原本我们想要的那个参考值”。

我们现在手头拥有的东西就两样: 输入给系统的参考值, 还有系统输出的数值, 可以简单写成一个函数表达式

$$out = f(ref)$$

为了让 out 变成我们想要的 ref , 我需要对这个式子进行改造:

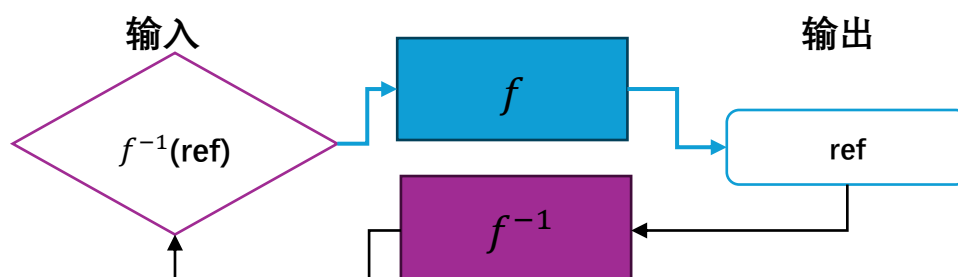
$$out = f(f^{-1}(ref))$$

现在再看这个式子, 按照式子计算 out 会等于什么? 很明显, 这次原本 out 的位置算出来的量变成了我们想要的 ref , 所以也就是说我们只需要往系统里输入一个参考值 $f^{-1}(ref)$, 最后就能输出得到 ref , 而这个也就是前面一直讲到的“校准”。

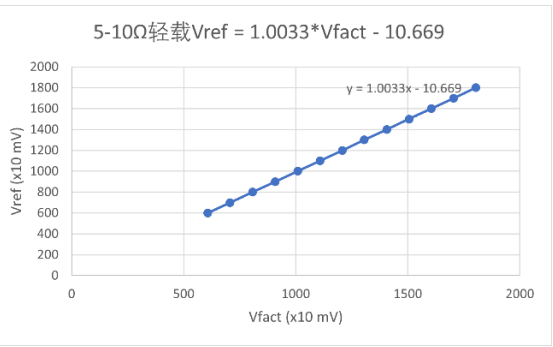
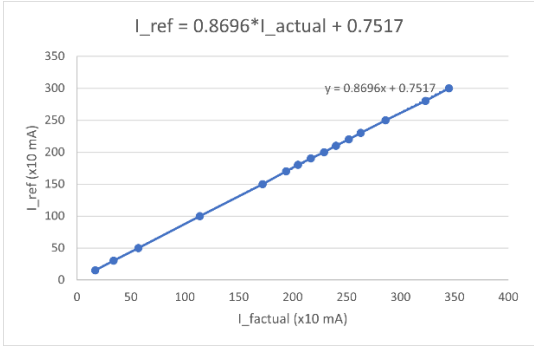
那这个 f^{-1} 要怎么获得呢?

前面我们说了, 现在手头就只拥有两样东西, 我们自己定的, 输入给 f 的 ref , 和 f 输出给我们的 out , 我们只需要输入多个不同的 ref 给系统 f , 然后记录多次 f 的输出值 out , 就可以获取非常多的 (ref, out) 数据点, 如果数据点写成 (ref, out) 这样的形式, 那毫无疑问, 对应的函数应该是 $out = f(ref)$, 那 f^{-1} 呢? 我们只需要把数据点写成这样 (out, ref) , 这一次写出来的函数是 $ref = g(out)$, 很显然 $g = f^{-1}$ 。

也就是说, 我们只需要记录输入和输出的数据点, 然后交换他俩的位置, 再把数据点放在 $x-y$ 平面上构建出函数, 就可以获得 f^{-1} 了。



输出校准



OLED 显示校准

